

Instructions: You have 110 minutes to complete the exam. **Please present your ID when turning in your exam paper.**

Be sure to show complete answers and all appropriate work **on the exam paper**; no scratch paper will be collected. You may use the back of any page (including this cover sheet) if you need more space. When continuing onto another page, be sure to **label the question number** on your work.

You are permitted your own paper notes, plus printouts of any course materials provided by the instructor (e.g. slides, assignments). During the exam, use of any electronic devices or other non-approved resources is prohibited, as is communicating with anyone other than the instructional staff. Violations of exam policies are grounds for failure of the course.

This examination consists of problems numbered 1-6; be sure to read all questions before beginning your exam. The boxes next to each question are for the grader to record your score; please leave them blank.

(1) The closed hashtable shown at right uses quadratic probing, with hash function $h(k) = k \bmod 16$ and probe function $p(i) = i + i^2$.

(a) [5 points] The value 18 is added to the hash table. Which slots in the hash table does it try to go into? Which box does it eventually land in? Show your work. (For reference, the first few multiples of 16 are 16, 32, 48, 64, 80.)

(b) [5 points] Explain why $p(i)$ is a poor choice for a quadratic probe function.

(a) $h(18) = 2$, so the first attempt is box 2.

$p(1) = 2$, and $2 + 2 = 4$, so the next attempt is box 4.

$p(2) = 6$, and $2 + 6 = 8$, so the next attempt is box 8.

$p(3) = 12$, and $2 + 12 = 14$. Since box 14 is empty, that is the final placement.

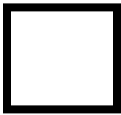
Grading notes: Initial hash is worth 1 point; correct probing is worth 4.

In general, a single arithmetic error should be -2 (so 3/5 if everything else is right).

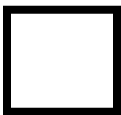
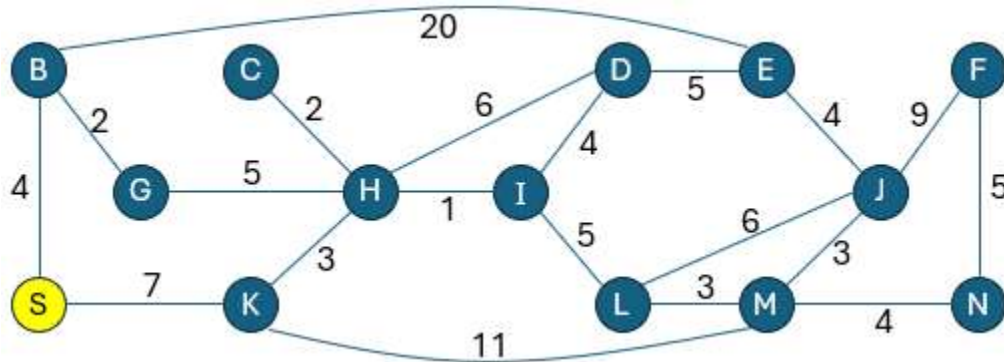
(b) $p(i)$ is always even for any integer i . Since the size of the table is also even, that means that the probe function will not visit every hash bucket.

Grading notes: 2/5 if they state the probe function doesn't hit all buckets without explaining how they know that. 4/5 if they don't bring up the size of the hash table in an otherwise correct explanation.

0	
1	97
2	34
3	
4	68
5	
6	
7	
8	20
9	53
10	
11	
12	
13	45
14	
15	

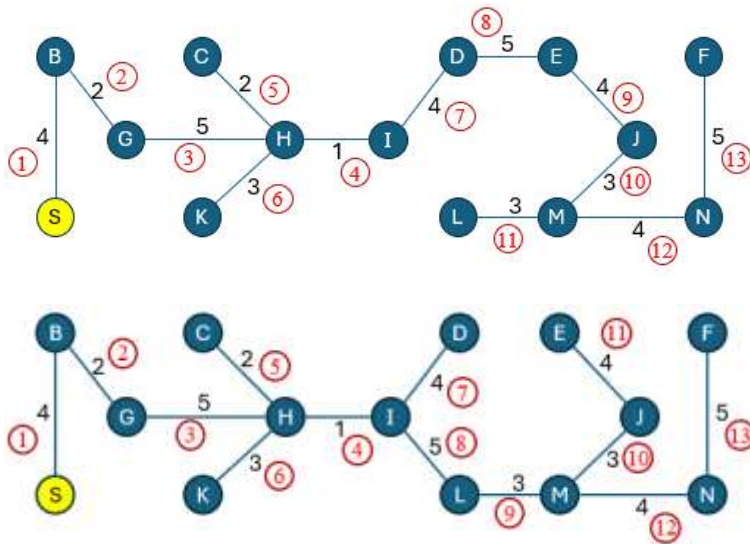


(2) (a) [8 points] Execute Prim's algorithm on the graph below starting from S, making sure to indicate clearly the order in which edges are added to the tree. (Please draw your own tree rather than marking up the graph – that can be very hard to grade.)



(b) [10 points] Let e_{min} be the uniquely lowest-weight edge in a graph G , and let T be any minimum spanning tree of G . Without relying on the correctness of any MST algorithm (i.e. you can't just say "Kruskal's algorithm always picks it"), prove that T must include e_{min} .

(a) Solution (either one acceptable – depends on how the tie between D-E and I-L was broken:



Grading notes: E & N in the second solution can have the order swapped and still be correct. Maximum score of 2/8 if the graph presented is not a tree. 2/8 if they run Kruskal's algorithm instead of Prim's.

Otherwise, -1 for each pair of edges with their order swapped (e.g. H-C is listed before H-I); -2 for each edge that they include that is not part of the MST.

(b) Several different approaches possible. The most direct is an exchange argument / proof by contradiction.

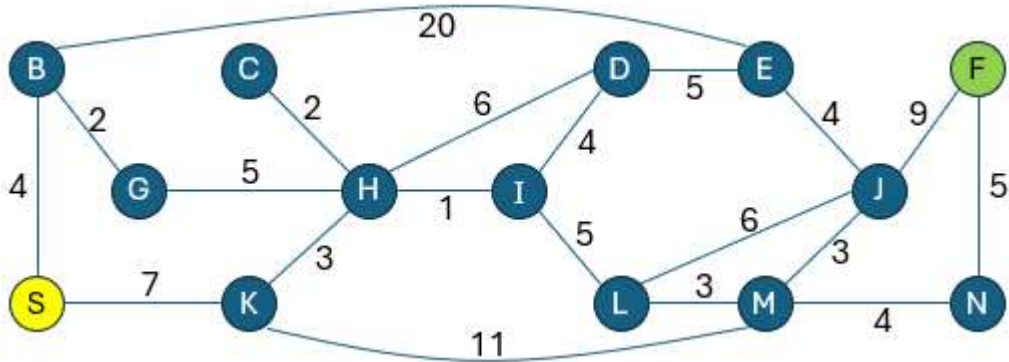
Let v_1, v_2 be the endpoints of e_{min} . Assume that T is an MST for G that does not include e_{min} . Since T is connected, there must be a path P from v_1 to v_2 in T . Then $P \cup \{e_{min}\}$ is a cycle in G . We can remove any other edge $e' \neq e_{min}$ from this cycle and get a new spanning tree T' that includes e_{min} . Since we are given that $e_{min} < e'$, the total weight of T' must be strictly less than T , contradicting the assumption that T is an MST.

Grading notes: Since students may present a variety of different proofs, fall back on the standard rubric: 8/10 for something that has a minor conceptual error, 5/10 for something that has a major conceptual error, 2/10 for having something at least vaguely on a correct path. For the particular exchange argument given in this solution, a common error will be to pick *any* edge of T to replace (which might disconnect the graph!); this is a 2-point error.



(3) (a) [10 points] Execute Dijkstra's algorithm to find the lowest-weight path from S to F in the graph given below. **Clearly show your work** so that it is obvious to anyone reading the order in which you explored the edges and finalized vertices.

Your final answer must include the actual path from S to F and its weight.



(b) [10 points] Construct a small graph containing at least one negative edge but no negative cycle on which Dijkstra's algorithm fails to find the lowest-weight path for some pair of nodes. Be sure to state the nodes in question, the path found by Dijkstra, and the optimal path.

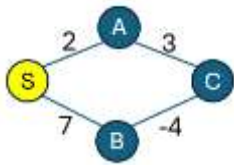


(c) [5 points] Give a clear English explanation why Dijkstra's algorithm cannot generally handle negative edges. (You may refer to your example, but you need more words besides that.)

S:0,--
B:4,S
K:7,S
G:6,B
E:24,B 20,D
H:11,G 10,K
M:18,K
I:11,H
C:12,H
D:16,H 15,I
L: 16,I
J:22,L 21,M
N:22,M
F:30,J 27,N

S:0,--
B:4,S
G:6,B
K:7,S
H:10,K
I:11,H
C:12,H
D:15,I
L:16,I
M:18,K
E:20,D
J:21,M
N:22,M
F:27,N

The path is $S - K - M - N - F$, with length 27.



(b) When calculating the path from S to C, Dijkstra will finalize A then C before it even explores B. It will find S-A-C with distance 5; the best path is S-B-C with distance 3.

(c) It assumes that path lengths only ever increase. Thus, it finalizes nodes before seeing all paths that lead to them – as soon as a vertex v comes off the priority queue, it assumes that the best path leading up to v has already been found. Negative edges decrease path length, violating that assumption.

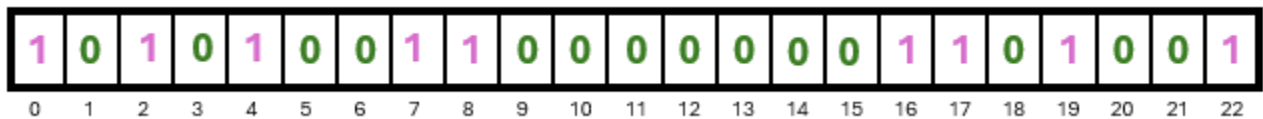


(4) (a) [10 points] A 23-bit Bloom filter is equipped with four hash functions: $h_1(k) = k \bmod 23$, $h_2(k) = (2k + 11) \bmod 23$, $h_3(k) = (3k - 5) \bmod 23$, $h_4(k) = (4k + 3) \bmod 23$.

(For reference, the first ten multiples of 23 are 23, 46, 69, 92, 115, 138, 161, 184, 207, 230.)

In the boxes above, show the status of the (initially empty) Bloom filter after the values 30, 40, and 50 are added to it. Show your work.

$$\begin{aligned}
 h_1(30) &= 7, h_2(30) = 2, h_3(30) = 16, h_4(30) = 8 \\
 h_1(40) &= 17, h_2(40) = 22, h_3(40) = 0, h_4(40) = 2 \\
 h_1(50) &= 4, h_2(50) = 19, h_3(50) = 7, h_4(50) = 19
 \end{aligned}$$



Grading nodes: -1 per incorrect bit

(b) [8 points] A similar Bloom filter (same size & hash functions) has had 10 unknown positive integers added to it, leaving it in the state shown at the bottom of the page.

Calculate the probability that an arbitrary positive integer that is **NOT** in this Bloom filter produces a false positive. Justify your answer.

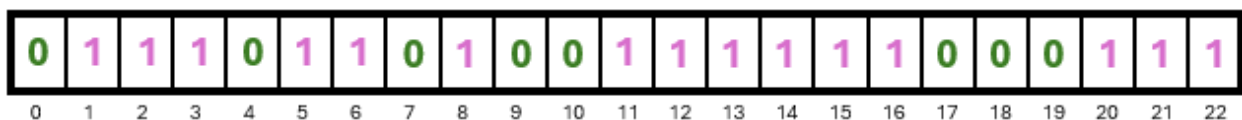
The Bloom filter has 15 bits that are 1s, out of 23 total slots.

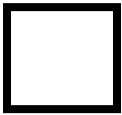
For an arbitrary integer, the probability that any individual hash function lands on a slot that has a 1 in it is therefore $15/23 \approx 0.652$.

We will only get a false positive if all four hash functions land on 1s.

Thus, the correct answer is $(15/23)^4 \approx 0.181$.

Grading notes: Obviously, students are not expect to produce the decimals – they are just there to provide a sense of scale. Basically, 4 points for recognizing that $15/23$ was the important fraction, and 4 points for recognizing that we needed to raise that number to the fourth power.





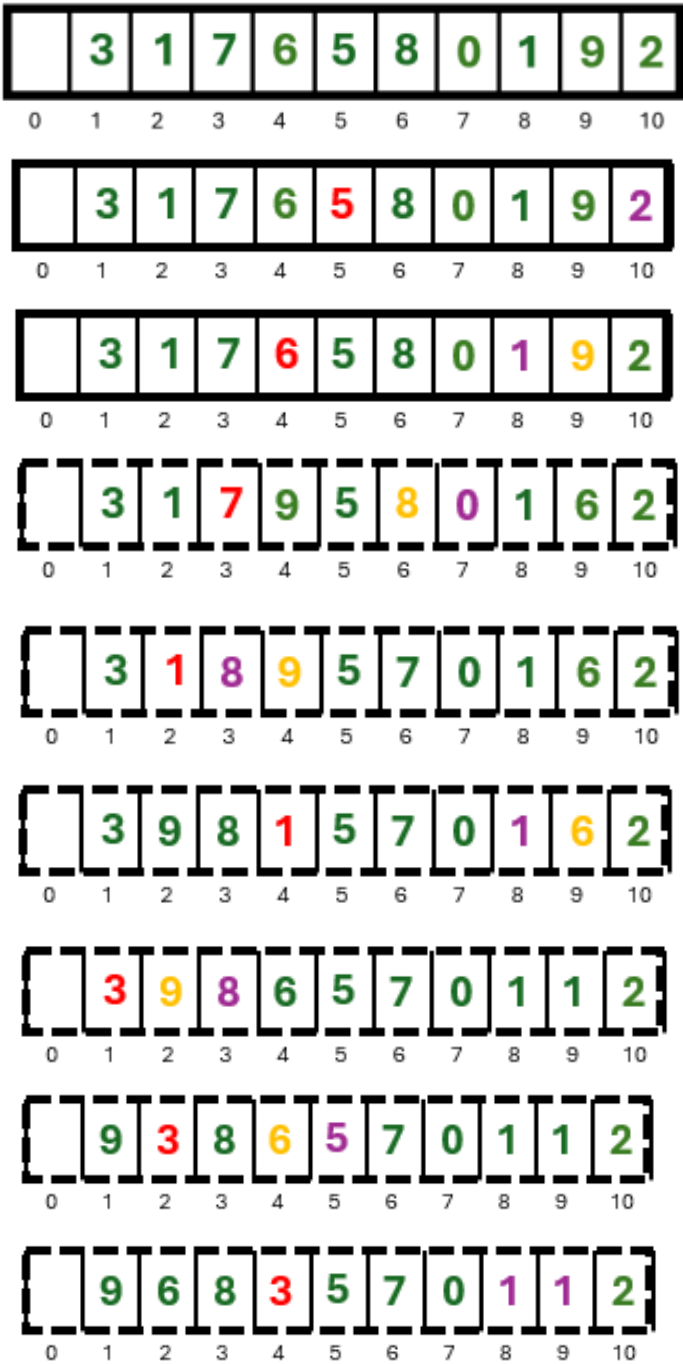
(5) [10 points] Given the array shown below, demonstrate the execution of the linear-time (“bottom-up”) process for turning it into a heap. You should redraw the array each time two elements swap positions.

Remember: parent node n has child nodes $2n$ and $2n + 1$

Elements #1-5 are parents; elements #6-10 are leaves. The heapify operation pushes down parent nodes as needed.

Extra copies of the array included here to show which elements were are looking at for each step. The only ones required for student answers are the ones with dashed borders (and obviously colors are not required).

Grading notes: 2/10 if they used the repeated insertion approach instead.
5/10 if they start at the top of the heap and work their way down.
Otherwise, -2 for an incorrect swap.
Max score 5/10 if their final result is not a valid heap.



(6) A very rural area has a long straight road with n scattered houses that are various distances apart (some neighbors may be fairly close, others may be miles apart). Let these be in order $h_1, h_2, \dots, h_n$, and their locations (measured in miles from the start of the road) be $d_1, d_2, \dots, d_n$.

Cell towers are going to be installed along this road to provide all n houses with cell service; a house must be within r miles of a tower to have service. For cost reasons, we want to build as few towers as possible to accomplish the goal of serving all n houses.

(a) [10 points] Give a greedy algorithm to solve this problem. Give the asymptotic runtime of your algorithm, and justify your response.

(b) [10 points] Prove that your algorithm always produces the optimal answer. (Note: This doesn't apply to everybody, but most of you need more explanation in your proofs.)

(a) Go to the first house you come to, then travel an additional r miles and put your tower there. Go r more miles, ignoring any houses you pass. Then repeat the process. (You could do this from the other end just as easily.)

More formally: Let d_{min} be the smallest remaining house location. Place a tower at location $d_{min} + r$. Remove all houses with locations $\leq d_{min} + 2r$ from consideration – they are covered. Repeat these steps until there are no more houses left to cover.

(b) The most direct approach is a “gets ahead, stays ahead” argument. Let S be our solution, and S' be any other solution. Both solutions must have a tower that covers h_1 . Perforce, our solution covers the maximum number of other houses along with h_1 – that's the greedy part of our approach. So after 1 tower, S has covered at least as many houses as S' .

We can apply the same argument to house h_i , where h_i is the house for our second tower. Of all solutions covering h_1 and h_i , no solution can possibly cover more houses. (Note that S' 's second tower might cover more houses than S 's second tower. But these “extra” houses were the ones that S' first tower didn't get.) This applies inductively to the rest of the solution.

Grading notes: Again, lots of possible solutions here, for both parts. Apply the general rubric.